

DDD-Enforcer: An AI-Powered Multi-Agent System for Real-Time Domain-Driven Design Enforcement

Ahmet Baran Dinçoguz
Dept. of Software Engineering
TED University
Ankara, Türkiye
baran.dincoguz@tedu.edu.tr

Ali Kendir
Dept. of Software Engineering
TED University
Ankara, Türkiye
ali.kendir@tedu.edu.tr

Murat Karakaya
Dept. of Software Engineering
TED University
Ankara, Türkiye
murat.karakaya@tedu.edu.tr

Abstract—Domain Model Degradation increases architectural technical debt by violating ubiquitous language and bounded contexts. This paper presents DDD-Enforcer, a VS Code extension that enforces DDD principles through real-time analysis combining Abstract Syntax Tree (AST) inspection with Large Language Models (LLMs). The system employs a four-stage multi-agent architecture to extract domain models from SRS-related documents and integrates a Retrieval-Augmented Generation (RAG) pipeline for requirement traceability. Experiments demonstrate 100% detection accuracy across 15 violation cases with an average latency of 4.49 seconds. The RAG component achieves 76.8% Top-1 accuracy, remaining robust under noisy conditions. Additionally, preliminary results for the MVC Test Orchestrator suggest potential for automated testing workflows. These findings confirm that hybrid LLM-static analysis effectively maintains DDD compliance.

Keywords—Domain-Driven Design, Software Architecture, Large Language Models, Multi-Agent Systems, Retrieval-Augmented Generation, Static Analysis, VS Code Extension, Technical Debt

I. INTRODUCTION

Domain-Driven Design (DDD), introduced by Eric Evans in 2003 [1], provides a powerful framework for tackling complexity in software development by aligning code structure with business domains. The DDD approach has gained significant traction, with the global DDD market in banking solutions alone valued at \$2.4 billion in 2024 and projected to reach \$8.7 billion by 2033, growing at a CAGR of 15.2% [2]. However, maintaining DDD principles across large codebases remains challenging, particularly as development teams scale and AI coding assistants become increasingly prevalent.

The consequences of architectural violations are severe. Research indicates that enterprises face a \$1.52 trillion software technical debt crisis globally, with Architectural Technical Debt (ATD) emerging as the most damaging category [4]. Organizations report that technical debt amounts to up to 40% of their entire technology estate, with 51% of companies allocating over 25% of their annual IT budgets to remediation efforts [5]. The problem is particularly acute in organizations with monolithic architectures, where 57% allocate over a quarter of their IT budget to technical debt remediation.

Traditional static analysis tools like ESLint, Pylint, and ArchUnit focus on syntax, style, and structural rules but lack semantic understanding of domain concepts. While ArchUnit

provides architecture testing through Java unit tests [6], and NDepend offers comprehensive .NET analysis through LINQ-based queries [7], neither leverages natural language understanding to detect subtle violations of ubiquitous language or context boundary semantics.

This paper introduces DDD-Enforcer, an intelligent tool that addresses Domain Model Degradation through a hybrid approach combining static analysis with AI-powered semantic understanding. The system operates as a real-time linter within VS Code, validating code against domain rules defined in a structured domain model and providing immediate feedback as developers write code.

The key contributions of this work include:

- A four-stage multi-agent architecture (Scout, Architect, Specialist, Synthesizer) for automatic domain model extraction from SRS-related documents using Google Gemini
- Integration of Large Language Models with AST analysis for intelligent violation detection across six violation types
- A Retrieval-Augmented Generation (RAG) pipeline using ChromaDB for source traceability, linking violations to original requirements
- A VS Code extension providing real-time diagnostics with “View Source” quick-fix actions
- Pydantic-enforced structured outputs ensuring consistent and reliable LLM responses

II. BACKGROUND

A. Domain-Driven Design Principles

Domain-Driven Design provides strategic and tactical patterns for managing software complexity. Strategic patterns include Bounded Contexts, which divide large systems into coherent areas with their own models and terminology, and Context Mapping, which defines relationships between contexts. Tactical patterns include Entities, Value Objects, Aggregates, Domain Events, and Repositories [1], [8].

A key DDD principle is the Ubiquitous Language—a shared vocabulary between developers and domain experts that must be consistently reflected in code. Violations occur when code uses synonyms (e.g., “Client” instead of “Customer”), generic terms (e.g., “Manager”, “Helper”), or crosses bounded context boundaries inappropriately.

As a general principle established by Microsoft's architectural guidance, a microservice should be “no smaller than an aggregate and no larger than a bounded context” [9]. With widespread microservices adoption and 92% of survey respondents reporting at least some success with the approach [10], enforcing DDD principles has become increasingly critical.

B. Large Language Models for Code Analysis

The widespread adoption of AI coding assistants has introduced significant challenges to codebase maintainability. Analysis of 211 million lines of code reveals that in 2024, the amount of copy-pasted code exceeded refactored (“moved”) lines for the first time on record. Furthermore, the frequency of duplicated code blocks has increased approximately 10 times compared to 2022 levels, accompanied by a 20–25% rise in churn rates for newly added code [11]. This shift suggests that AI tools optimize for short-term output at the expense of long-term architectural integrity.

Recent advances in LLM capabilities have enabled structured output generation, where models return responses conforming to specified JSON schemas. Google Gemini's structured output feature, combined with Pydantic validation, ensures reliable, parse-able responses—critical for automated tooling [12].

C. Retrieval-Augmented Generation

RAG optimizes LLM output by referencing authoritative knowledge bases before generating responses [13]. The field has seen explosive growth, with over 1,200 RAG-related papers published on arXiv in 2024, compared to fewer than 100 in 2023 [14]. Key advances include Self-RAG (models deciding when and how much to retrieve), multi-hop retrieval for complex queries, and Graph-RAG for knowledge graph extraction [15]. For code analysis applications, RAG enables grounding LLM responses in project-specific documentation, reducing hallucination and providing traceable source references. Vector databases like ChromaDB (market valued at \$2.2 billion in 2024, projected to reach \$10.6 billion by 2032 [16]) provide the infrastructure for semantic similarity search.

III. SYSTEM ARCHITECTURE

A. Overview

DDD-Enforcer consists of three main components: a Python FastAPI backend [22] for code analysis and LLM orchestration, a RAG pipeline for source traceability, and a TypeScript VS Code extension for developer interaction.

TABLE I
DDD-ENFORCER TECHNOLOGY STACK

Component	Technology	Version
Web Framework	FastAPI	0.124.0
Data Validation	Pydantic	2.12.5
LLM Integration	Google Generative AI	0.8.5
Vector Database	ChromaDB	1.3.5
Embeddings	Sentence Transformers	5.1.2
PDF Parsing	pypdf	latest
DOCX Parsing	python-docx	latest

Listing 1. Pydantic Response Schema

```
class Violation(BaseModel):
    type: Literal["SynonymViolation",
                 "BannedTermViolation", ...]
    message: str
    suggestion: str

class ValidationResponse(BaseModel):
    is_violation: bool
    violations: List[Violation]
```

B. Multi-Agent Pipeline for Domain Model Generation

The domain model generation employs a four-stage agentic architecture, inspired by recent advances in multi-agent systems for software engineering [17]. Unlike monolithic LLM approaches, this pipeline decomposes the complex task of domain extraction into specialized stages:

Stage 1 - Scout Agent: Extracts domain-relevant sentences from SRS-related documents using smart chunking (10,000-character chunks with sentence boundary preservation). The Scout focuses on business entities, rules, workflows, and relationships, filtering out boilerplate content.

Stage 2 - Architect Agent: Analyses extracted sentences to identify distinct Bounded Contexts. A well-formed context exhibits its own terminology, entities, rules, and logical team ownership. The agent typically identifies 2-8 contexts per project.

Stage 3 - Specialist Agent: Performs deep analysis of each context in a single optimized API call, extracting Aggregate Roots and Entities with their attributes, Value Objects with their component attributes, Domain Events and Business Rules, synonyms to avoid for each entity, and context dependencies that define allowed interactions between bounded contexts.

Stage 4 - Synthesizer Agent: Merges all context analyses into a cohesive `DomainModel` JSON structure, resolving duplicate entities, maintaining naming consistency, and generating the final validated output using Pydantic schemas.

The pipeline implements rate limiting (6-second minimum between API calls) and exponential backoff for quota management, ensuring reliable operation within API constraints.

C. Code Parsing and AST Analysis

The extension captures content when a developer saves a Python file and sends it to the backend for AST analysis through a custom `DomainVisitor` class that extracts class definitions with base classes and methods, import statements covering both import and from imports, function definitions with parameters and type hints, variable assignments for Value Object violation detection, and function calls for Domain Event analysis.

This AST-based approach provides safe analysis without code execution, extracting structural information that informs the LLM's semantic analysis.

D. LLM-Powered Violation Detection

The structural AST data, combined with domain rules from `model.json`, is transmitted to Google Gemini for semantic analysis. The LLM evaluates the code against six distinct violation types, including Synonym, Banned Term, and Context Boundary violations, ensuring strict adherence to DDD principles. Responses are returned as Pydantic-enforced structured outputs, as illustrated in Listing 1.

E. RAG Pipeline for Source Traceability

A key innovation of DDD-Enforcer is the RAG pipeline that links violations to their source in the original SRS-related documents. The pipeline uses ChromaDB with the `all-MiniLM-L6-v2` embedding model and implements:

Section-Aware Chunking: Documents are split by structural headers (numbered sections and markdown headers), with a configurable chunk size (default 1000 characters) and overlap (default 200 characters). Large sections are split at paragraph boundaries to avoid mid-sentence breaks.

Minimal Metadata Storage: Each chunk stores only structural metadata—document ID, document name, section name/number, and estimated page number. The pipeline has zero domain knowledge and doesn't extract entities, synonyms, or banned terms.

Semantic Query Matching: Queries are built by extracting quoted terms from violation messages and combining them with the full violation description. The embedding model handles semantic matching without hardcoded query strategies.

When a violation is detected, the RAG pipeline retrieves the top-3 most relevant source chunks, providing developers with direct links to the requirements that define the violated rule.

IV. IMPLEMENTATION

A. Backend Components

The backend is implemented in Python 3.10+ using the technology stack shown in Table I.

Key modules include:

- `architect.py` (701 lines): Multi-agent pipeline with rate limiting and exponential backoff
- `llm_client.py` (261 lines): Gemini integration with structured output enforcement
- `parser.py` (181 lines): AST-based code analysis with `DomainVisitor` class
- `document_parser.py` (64 lines): Multi-format SRS-related document parsing (PDF, DOCX, TXT)
- `rag_pipeline.py` (473 lines): ChromaDB-based source retrieval with intelligent chunking
- `schemas.py` (117 lines): Pydantic models for Entity, ValueObject, DomainEvent, BoundedContext, and DomainModel

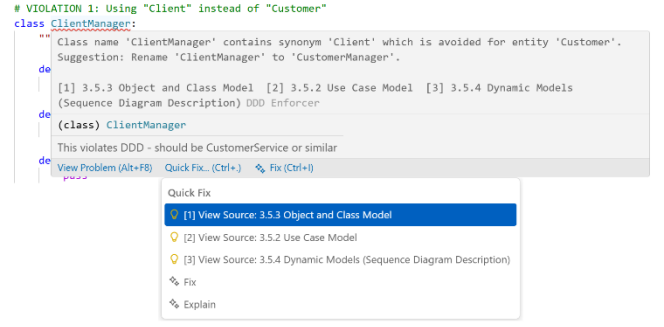


Fig. 1. DDD-Enforcer VS Code extension showing violation diagnostics with source references.

B. VS Code Extension

The extension is implemented in TypeScript 5.9.3 using the VS Code Extension API [23]. Key features include:

File Save Event Handling: The extension subscribes to `onDidSaveTextDocument` events, filtering Python files and triggering validation.

Diagnostic Collection: Violations are displayed using VS Code's `DiagnosticCollection`, with intelligent keyword location finding that prioritizes class and function definitions.

Code Actions: A `CodeActionProvider` offers “View Source” quick fixes that open the relevant SRS-related document section, leveraging the RAG pipeline's source references.

Configuration: The extension uses Axios for HTTP communication with the backend, with proper error handling for server unavailability.

C. CI/CD Pipeline

Automated quality assurance is implemented through GitHub Actions:

Backend CI: Triggers on push/PR to main branch for backend changes, running Python 3.11 syntax checking and pytest tests.

Extension CI: Compiles TypeScript, runs ESLint checks, and validates extension packaging.

V. EVALUATION

A. Experimental Setup

DDD-Enforcer was evaluated using a sample e-commerce domain with five bounded contexts: Customer Management, Product Catalog, Order Processing, Inventory Management, and Payment Processing. The SRS document comprised approximately 5,600 characters describing business rules, entity relationships, and context boundaries.

TABLE II
VIOLATION DETECTION RESULTS

Violation Type	Test Cases	Detected
SynonymViolation	5	5 (100%)
BannedTermViolation	4	4 (100%)
NamingConvention	3	3 (100%)
ContextBoundary	3	3 (100%)
Total	15	15 (100%)

B. Domain Model Generation

The multi-agent pipeline successfully processed the SRS-related document, demonstrating:

- **Context Identification:** Correctly identified all 5 bounded contexts with appropriate names and descriptions
- **Entity Extraction:** Extracted 15+ entities with attributes and synonym lists
- **Dependency Mapping:** Correctly inferred allowed dependencies between contexts
- **API Efficiency:** Completed generation in 4-8 API calls through intelligent batching

C. Violation Detection

Table II summarizes the violation detection results on test files containing intentional violations.

Example detections include:

- **ClientManager** flagged for using “Client” (synonym for Customer) and “Manager” (banned global term)
- **Purchase, Transaction, Item** correctly identified as synonym violations
- Cross-context imports detected when Payment Processing accessed Inventory Management without declared dependency

D. RAG Source Retrieval

We evaluated source traceability across banking (D1), healthcare (D2), and e-commerce (D3) domains on an AMD Ryzen 6900S/16GB RAM system (5 iterations/query). To test robustness, D3 was expanded into mildly (D3-MP, 34 chunks) and extremely polluted (D3-EP, 40 chunks) variants against a 14-chunk baseline, simulating realistic SRS-related document noise. Test cases were LLM-generated and manually verified. As shown in Table III, the system achieved 76.8% Top-1 and 88.8% Top-3 accuracy with an average latency of 140.52 MS, confirming suitability for real-time IDE integration.

TABLE III
RAG RETRIEVAL PERFORMANCE ACROSS DOMAIN DATASETS

Dataset	Chunks	Avg Time	Top-1	Top-3
D1 (Banking)	15	134.41 MS	72.0	84.0
D2 (Healthcare)	30	134.07 MS	80.0	84.0
D3 (E-commerce)	14	158.23 MS	80.0	92.0
D3-MP (Mild)	34	141.31 MS	76.0	92.0
D3-EP (Extreme)	40	134.53 MS	76.0	92.0
Average	-	140.52 MS	76.8	88.8

Notably, the pollution experiments revealed remarkable robustness: Dataset D3-EP, despite containing $2.86\times$ more chunks with substantial noise injection, maintained identical Top-3 accuracy (92%) compared to the clean baseline. Top-1 accuracy decreased modestly from 80% to 76%, indicating that while the correct section remained within the top 3 results, noise occasionally pushed it from the first position.

An analysis of retrieval failures revealed that **ContextViolation** types (architectural boundary violations) occasionally retrieved generic architecture sections rather than the specific context definition. This suggests that future improvements should focus on enhanced metadata tagging for bounded context relationships, potentially through graph-based retrieval strategies.

E. Performance Metrics

To evaluate system performance, we conducted benchmarks across varying input sizes for both domain model generation and code validation. Notably, we employed a tiered model strategy: Gemini 2.5 Flash for the computationally intensive domain model generation pipeline, and Gemini 2.5 Flash Lite for real-time code validation. This design decision was motivated by preliminary experiments showing that Flash Lite provides comparable violation detection accuracy at significantly reduced cost and latency-critical factors for responsive IDE integration.

Table IV presents the domain model generation benchmarks. The multi-agent pipeline completed processing in 90-149 seconds depending on document complexity, with costs scaling linearly with token count. The non-monotonic latency pattern reflects the batch optimization in the Specialist agent, where larger documents benefit from consolidated API calls.

Table V shows validation performance across code file sizes. The average validation latency of 4.49 seconds provides acceptable responsiveness for on-save validation workflows.

TABLE IV
DOMAIN MODEL GENERATION PERFORMANCE

Document Size	Chars	Tokens	Latency
Small	864	216	111.84 s
Medium	1,800	450	90.32 s
Large	2,666	666	149.04 s

TABLE V
CODE VALIDATION PERFORMANCE

Code Size	Chars	Tokens	Latency
Small	608	183	3.19 s
Medium	1,110	309	4.89 s
Large	2,019	583	5.96 s
Average	1,246	358	4.49 s

VI. RELATED WORK

A. Architecture Enforcement Tools

ArchUnit provides Java architecture testing through unit test assertions [6]. NDepend uses LINQ-based queries for .NET static analysis with comprehensive metrics [7]. NetArchTest provides similar capabilities for .NET Core. However, these tools require manual rule definition and lack semantic understanding of domain concepts.

B. AI-Assisted Code Analysis

GitHub Copilot and Amazon CodeWhisperer provide AI-powered code completion, with Copilot achieving 65.2% correctness rates in recent evaluations [18]. However, these tools focus on code generation rather than architectural compliance. Recent work on “AI coding assistants” has shown convergent evolution toward multi-prompt reasoning and external tool calling [19], aligning with DDD-Enforcer's approach.

C. RAG for Software Engineering

Red Hat's research highlights RAG as a “game-changer in code generation,” automating retrieval of documentation, code repositories, and research papers [20]. Azure AI Search provides enterprise RAG infrastructure [21]. DDD-Enforcer extends these concepts specifically for DDD compliance with domain model grounding.

D. Multi-Agent Systems

Frameworks like LangGraph, AutoGen, and CrewAI have matured for multi-agent orchestration [17]. LangGraph's graph-based approach using DAGs enables complex, multi-step workflows with clear visualization. DDD-Enforcer's four-stage pipeline draws from these patterns while specializing for domain model extraction.

TABLE VI
LLM PRICING AND OPERATIONAL COSTS

Cost (Per 1 Million Tokens)	Flash	Flash Lite
Input	\$0.30	\$0.10
Output	\$2.50	\$0.40
Use Case	Generation	Validation
Avg. cost per operation	\$0.019	\$0.00076

VII. DISCUSSION

A. Strengths

DDD-Enforcer offers distinct advantages over rule-based tools. By leveraging LLM-driven semantic analysis, it detects subtle violations—such as inappropriate synonyms—that static analyzers miss. The system lowers DDD adoption barriers by automatically extracting structured domain models from documentation, eliminating manual creation. RAG integration ensures accountability via requirement-level traceability, explaining the rationale behind violations. Furthermore, real-time IDE feedback shifts validation left, catching errors during active development rather than in CI/CD pipelines. Finally, Pydantic ensures structured, reliable LLM outputs for consistent validation.

B. Limitations

Current limitations include restriction to Python, with support for TypeScript, Java, and C# planned. Reliance on LLM APIs necessitates internet access and incurs operational costs. While accurate, the system may occasionally produce false positives for common terms in non-domain contexts, requiring developer review. Lastly, the quality of generated domain models is intrinsically tied to the completeness of the input SRS-related documentation.

C. Cost Analysis

DDD-Enforcer employs a tiered pricing strategy by utilizing different model tiers for different tasks. The domain model generation pipeline, which runs infrequently (typically once per project or when requirements change), uses Gemini 2.5 Flash for its superior reasoning capabilities. Real-time code validation, which is executed on every file save, uses Gemini 2.5 Flash Lite to minimize operational costs while maintaining acceptable accuracy.

Table VI summarizes the pricing structure and average operational costs. Based on our benchmarks, projected costs for a typical development workflow include domain model generation at \$0.008-\$0.033 per document on average as a one-time cost, per-validation costs averaging \$0.00076 using Flash Lite, and monthly estimates around \$22.74 assuming 1,000 validations per day.

These costs compare favourably to the 25%+ of IT budgets currently spent on technical debt remediation [5], particularly considering that early violation detection prevents the accumulation of architectural debt that becomes exponentially more expensive to resolve over time.

VIII. FUTURE WORK

Several enhancements are planned including multi-language support by extending parsers to Java, TypeScript, C#, and Go, implementation of quick fix actions for automatic code transformation to resolve violations, team collaboration features with shared domain models and version control integration, metrics dashboards for tracking DDD compliance trends over time, development of fine-tuned models through domain-specific training for improved accuracy, and Graph-RAG integration for knowledge graph extraction that can capture complex domain relationships and dependencies.

IX. CONCLUSION

DDD-Enforcer demonstrates that combining traditional static analysis with LLM intelligence provides an effective solution for enforcing Domain-Driven Design principles. The four-stage multi-agent architecture enables automatic domain model generation from natural language requirements, reducing the barrier to DDD adoption. Real-time validation through VS Code integration ensures immediate feedback, helping developers maintain architectural consistency as they write code.

In an era where AI coding assistants are widely adopted and enterprises face \$1.52 trillion in technical debt, tools like DDD-Enforcer become essential for maintaining software architecture integrity. The RAG-enhanced source traceability provides accountability, linking every violation to its origin in project requirements.

The system addresses a critical gap in software development tooling by providing semantic understanding of domain concepts—something traditional static analysis tools cannot achieve. As AI continues to transform software development, intelligent architecture enforcement will be key to managing complexity at scale.

This project's open-source implementation can be accessed via this GitHub repository [24].

ACKNOWLEDGMENT

This study was conducted as part of a course (SENG 472-Building AI Applications with Large Language Models) project under the supervision of Prof. Dr. Murat Karakaya, at TED University.

REFERENCES

- [1] E. Evans, "Domain-Driven Design: Tackling Complexity in the Heart of Software," Addison-Wesley, 2003.
- [2] MarketIntelto, "Domain-Driven Design in Banking Solutions MarketResearch Report 2033," 2024. [Online]. Available: <https://marketintelto.com/report/domain-driven-design-in-banking-solutions-market>
- [3] TrychAI, "AI Coding Tools Market Analysis 2024: Productivity Statistics," 2024. [Online]. Available: <https://www.trychai.io/blogs/ai-coding-tools-market-analysis-2024-productivity-statistics>
- [4] Architecture and Governance Magazine, "New Research: Architectural Technical Debt Most Damaging Amid \$1.52 Trillion Crisis," 2024.
- [5] vFunction, "How to Manage Technical Debt in 2025," 2024. [Online]. Available: <https://vfunction.com/blog/how-to-manage-technical-debt/>
- [6] ArchUnit, "Unit test your Java architecture," 2024. [Online]. Available: <https://www.archunit.org/>
- [7] K. Grzybek, "Modular Monolith: Architecture Enforcement," 2024. [Online]. Available: <https://www.kamilgrzybek.com/blog/posts/modular-monolith-architecture-enforcement>
- [8] V. Vernon, "Implementing Domain-Driven Design," Addison-Wesley, 2013.
- [9] Microsoft, "Tactical DDD for Microservices," Microsoft Learn, 2024. [Online]. Available: <https://learn.microsoft.com/en-us/azure/architecture/microservices/model/tactical-ddd>
- [10] M. Loukides and S. Swayer, "Microservices Adoption in 2020," O'Reilly Radar, July 2020. [Online]. Available: <https://www.oreilly.com/radar/microservices-adoption-in-2020/>
- [11] W. Harding, "AI Copilot Code Quality: Evaluating 2024's Increased Defect Rate via Code Quality Metrics," GitClear, Feb. 2025.
- [12] Google, "Gemini API Documentation," 2024. [Online]. Available: <https://ai.google.dev/docs>
- [13] AWS, "What is Retrieval-Augmented Generation?," 2024. [Online]. Available: <https://aws.amazon.com/what-is/retrieval-augmented-generation/>
- [14] arXiv, "A Systematic Review of RAG Systems," 2025. [Online]. Available: <https://arxiv.org/html/2507.18910v1>
- [15] Microsoft Research, "GraphRAG: Unlocking LLM Discovery on Narrative Private Data," 2024.
- [16] Aloa, "Pinecone vs Weaviate vs ChromaDB 2025," 2025. [Online]. Available: <https://aloe.co/ai/comparisons/vector-database-comparison>
- [17] Datagrom, "Top 3 Trending Agentic AI Frameworks: LangGraph vs AutoGen vs CrewAI," 2024.
- [18] arXiv, "Evaluating Code Quality: GitHub Copilot, CodeWhisperer, ChatGPT," 2024. [Online]. Available: <https://arxiv.org/html/2304.10778v2>
- [19] S. Lau and P. J. Guo, "The Design Space of LLM-Based AI Coding Assistants," Univ. of California San Diego, 2025.
- [20] Red Hat, "Redefining Development: RAG Revolution in Software Engineering," 2024.
- [21] Microsoft, "RAG in Azure AI Search," Microsoft Learn, 2024.
- [22] FastAPI, "FastAPI Documentation," 2024. [Online]. Available: <https://fastapi.tiangolo.com/>
- [23] Microsoft, "VS Code Extension API," 2024. [Online]. Available: <https://code.visualstudio.com/api>
- [24] A. B. Dinçoğuz, "DDD-Enforcer." GitHub repository. [Online]. Available: <https://github.com/barandincoguz/DDD-Enforcer>